Speed Limit Sign Detection and Driver Alert System

October 2025

By

Bogdan Petrescu

Student number 202333824

Word count: 1783

Contents

1. Project background and purpose

1.1. Introduction

This project aims to design and evaluate an AI-powered system that detects UK road speed limit signs in real time and alerts the driver if the detected limit differs from the official map speed or the vehicle's current speed. The solution will rely on computer vision to identify traffic signs, retrieve road speed data from a map service, and issue an alert when a temporary or lower speed limit is detected.

The system will run on a **mobile or embedded device** (e.g., smartphone or Raspberry Pi), with the final platform to be chosen after analysis of performance, power, and compatibility criteria. The prototype will be tested in controlled environments using printed or recorded sign data, avoiding public road testing.

1.2. Objectives

- **Primary objectives:**

- Detect UK circular speed limit signs (20–70 mph) using a YOLO-based AI model.
- Compare detected sign limits against live or cached map speed data.
- Alert the driver if a discrepancy or overspeeding condition occurs.
- Maintain real-time responsiveness (≤ 1 second total latency).

- **Secondary objectives:**

- Enable offline detection and GPS speed measurement.
- Log local results for testing and validation.
- Evaluate performance, accuracy, and energy efficiency across platforms.

1.3. Scope

This project covers software design, training, deployment, and testing of the detection and alert system. Testing will be confined to **controlled conditions** (lab or simulation).
The following are outside scope:

- Integration with real vehicles or on-road trials.
- Any driver-assist automation or enforcement functionality.

1.4. Deliverables

- Trained YOLO detection model for UK speed limit signs.

- Functional prototype app (mobile or embedded) performing detection, comparison, and alerting.
- Evaluation report with detection accuracy and performance metrics.
- Full project documentation, source code, and final report.

1.5. Constraints

- Dependence on external map APIs for comparison data.
- Hardware and OS platform finalised post-analysis.
- Continuous power assumed (no battery optimisation required).

1.6. Assumptions

- Public UK traffic datasets are suitable for model training.
- GPS and map data APIs are accessible and accurate.
- Selected hardware will support real-time inference.
- All tests will comply with University safety and ethics standards.

2. Project rationale and operation

2.1. Project benefits

- The system aims to reduce driver error by highlighting discrepancies between observed and official speed limits. It demonstrates real-world use of AI and computer vision in transport safety and showcases the feasibility of local, privacy-respecting inference on consumer or embedded devices.
  This aligns with wider goals in intelligent transport, automation safety, and edge-AI research.

2.2. Project operation

The project will use **iterative development** with biweekly sprints. Each sprint includes planning, implementation, testing, and review. The process flow is:

1. Dataset acquisition and pre-processing.
2. Model training and validation using YOLO.
3. Conversion to an embedded or mobile inference format.
4. Integration with GPS/map comparison logic and alert interface.
5. System testing, refinement, and documentation.

Requirements integration in Agile development

The functional and non-functional requirements defined in the ***Requirements Document*** act as the project's user stories and acceptance criteria. Each requirement will be implemented, tested, and reviewed iteratively through two-week Agile sprints. This allows continuous validation against user expectations and early identification of issues. At the end of each sprint, a demonstrable increment (e.g., functional detection module, GPS comparison, or alert system) will be reviewed to confirm that it satisfies the relevant requirement before progressing to the next stage.

2.3. Options

- Detection framework

YOLO is selected for its balance of detection accuracy (mAP ≈ 90–95 %) and high inference speed (30–60 FPS). Alternatives like Faster R-CNN were dismissed for slower performance unsuitable for real-time mobile or embedded operation.
The specific YOLO variant will be determined experimentally, based on accuracy, inference latency, and memory footprint.

- Accuracy target and reasoning

A **99 % accuracy goal** is set to ensure near-perfect detection reliability. A 90 % rate would imply misreading or missing 1 in 10 signs — unacceptable in a safety context. Achieving 99 % ensures false alerts remain rare and meaningful.

- Detection range analysis

Testing will be performed at **1–3 m distance**, replicating typical dashcam or phone-mount perspectives. This range ensures the model can read signs clearly without excessive computational scaling or image distortion.

- Platform flexibility

The app will initially be tested on standard smartphones for accessibility. Depending on hardware performance and power metrics, deployment may extend to other devices such as Raspberry Pi or Jetson Nano to evaluate portability and embedded feasibility.

2.4. Risk analysis and mitigation

Risk	Likelihood	Impact	Mitigation	Residual Effect / Rationale
Low detection accuracy under poor lighting	3	5	Add low-light and night images to dataset	Expected reduction to (1,3) after retraining
Model overfitting	2	4	Use data augmentation and cross-validation	Residual low (1,2)
Inference delay too high	3	4	Quantise and optimise model for target device	Residual (1,3)
GPS/map latency	2	3	Cache previous valid readings	Negligible impact
Privacy or data misuse	2	5	Ensure all data processed locally	Eliminated (0,0)
Schedule slippage	3	5	Two-week sprints and weekly progress tracking	Residual (1,3)
Legal liability for driver advice	1	5	State clearly “for demonstration use only”	Minimal risk post-mitigation

2.5. Resources required

Hardware:

- Smartphone or Raspberry Pi with camera and GPS.
- Laptop/desktop for training.

- Printed or on-screen UK speed limit signs.

Software:

- Python with YOLO framework.
- TensorFlow Lite or ONNX Runtime for deployment.
- OpenCV for pre-processing.
- Android Studio, Flutter, or equivalent IDE.
- Google Maps API or open-source equivalent.
- GitHub for version control.

All software is open-source or available under academic licence.

2.6. Ethical and legal considerations

- Testing involves no personal data collection.
- No footage or location data will leave the device.
- App labelled as “Advisory and Demonstration Only”.
- Complies with GDPR and University of Hull ethical guidelines.
- No road testing or user distraction during motion.
- All datasets will be open, publicly licensed, or anonymised.

2.7. Commercial considerations

The concept could evolve into a commercial driver-assist or telematics feature. Current navigation tools (e.g. Google Maps, Waze) rely solely on stored map limits. This project’s real-sign validation adds unique value for temporary or mis-mapped limits.
Prototype costs are minimal, relying on personal hardware and free datasets.

3. Project methodology and outcomes

3.1. Initial project plan

3.1.1. Tasks and milestones

Each development phase is aligned with Agile principles, with the defined requirements serving as user stories. Progress through each sprint will focus on delivering one or more functional features from the requirements document, followed by sprint reviews and testing to confirm requirement completion.

Phase	Timeline (2025–26)	Key Tasks	Deliverables
Phase 1 – Research & Documentation	Weeks 1–4 (Oct–Nov 2025)	Collect datasets, evaluate platforms, finalise requirements and ethics approval	Requirements Document, PDD
Phase 2 – Model Development	Weeks 5–10 (Dec–Jan)	Train YOLO model, perform augmentation, assess accuracy	Trained model + analysis
Phase 3 – System Integration	Weeks 11–16 (Feb–Mar)	Implement app/embedded logic, GPS comparison, alert system	Functional prototype
Phase 4 – Testing & Evaluation	Weeks 17–20 (Mar–Apr)	Conduct lab trials, analyse metrics	Test report
Phase 5 – Documentation & Submission	Weeks 21–24 (Apr–May)	Prepare final report and demo video	Final submission

3.1.2. Schedule Gantt chart

- Appendix a – Gantt chart

Duration: 24 weeks (Oct → May)
Sprint length: 2 weeks
Total: 12 sprints

Sprint	Dates (approx)	Goal / Theme	Key Tasks	Deliverables / Milestones
S1	W1–W2 (Oct 1–14 2025)	Project foundation & ethics	Finalise objectives, success metrics, and safety scope ("lab / sim only"). Draft ethics form.	Confirmed problem statement + ethics draft
S2	W3–W4 (Oct 15–28)	Dataset & Platform selection	Collect sample GTSRB / TSR-UK data, review hardware (Android vs RPi), create requirement backlog.	Dataset plan + PDD + Requirements Document approved
S3	W5–W6 (Nov 1–14)	Data prep & baseline model	Clean / augment dataset; train baseline YOLO; record accuracy & latency.	Baseline model + metrics report
S4	W7–W8 (Nov 15–28)	Accuracy & optimisation	Add night/low-light images; cross-validation; quantise model (TFLite/ONNX).	Edge-ready model (>95 % accuracy)
S5	W9–W10 (Dec 1–14)	Temporary-sign logic	Implement priority handling for temporary signs and retrain; final model validation.	Model v2 + Analysis Report
S6	W11–W12 (Dec 15–Jan 5)	Camera + UI pipeline	Integrate camera inference & on-screen detected speed overlay.	Live detection view (FR-01/02)
S7	W13–W14 (Jan 6–19)	GPS + map data integration	Build GPS and map API modules; auto-comparison engine.	Working GPS / map comparison (FR-03/04)
S8	W15–W16 (Jan 20–Feb 2)	Alert + offline modes	Implement buzzer/visual alerts, offline detection, caching logic.	Functional prototype (FR-05–FR-07)
S9	W17–W18 (Feb 3–16)	Privacy + UI polish	Disable data storage; add advisory warning; refine UI for usability.	Privacy-safe UI build (FR-08 – FR-10)
S10	W19–W20 (Feb 17–Mar 2)	Testing & evaluation	Create test harness; run lab tests (printed signs, lighting); collect stats.	Test Report draft + raw metrics
S11	W21–W22 (Mar 3–16)	Refinement & report writing	Fix performance bugs; compile evaluation results; start final report & demo script.	Release candidate build + Report outline
S12	W23–W24 (Mar 17–Apr 30)	Finalisation & submission	Record demo video; proofread documentation; supervisor review; submit.	Final submission + presentation ready

3.2. Project control

Progress will be monitored weekly with the supervisor. Code and data versions will be managed via GitHub. Each sprint will conclude with a retrospective to assess progress and redefine goals. Performance indicators: task completion rate, model accuracy progression, and adherence to schedule.

3.3. Project evaluation

The system will be evaluated using quantitative and qualitative criteria:

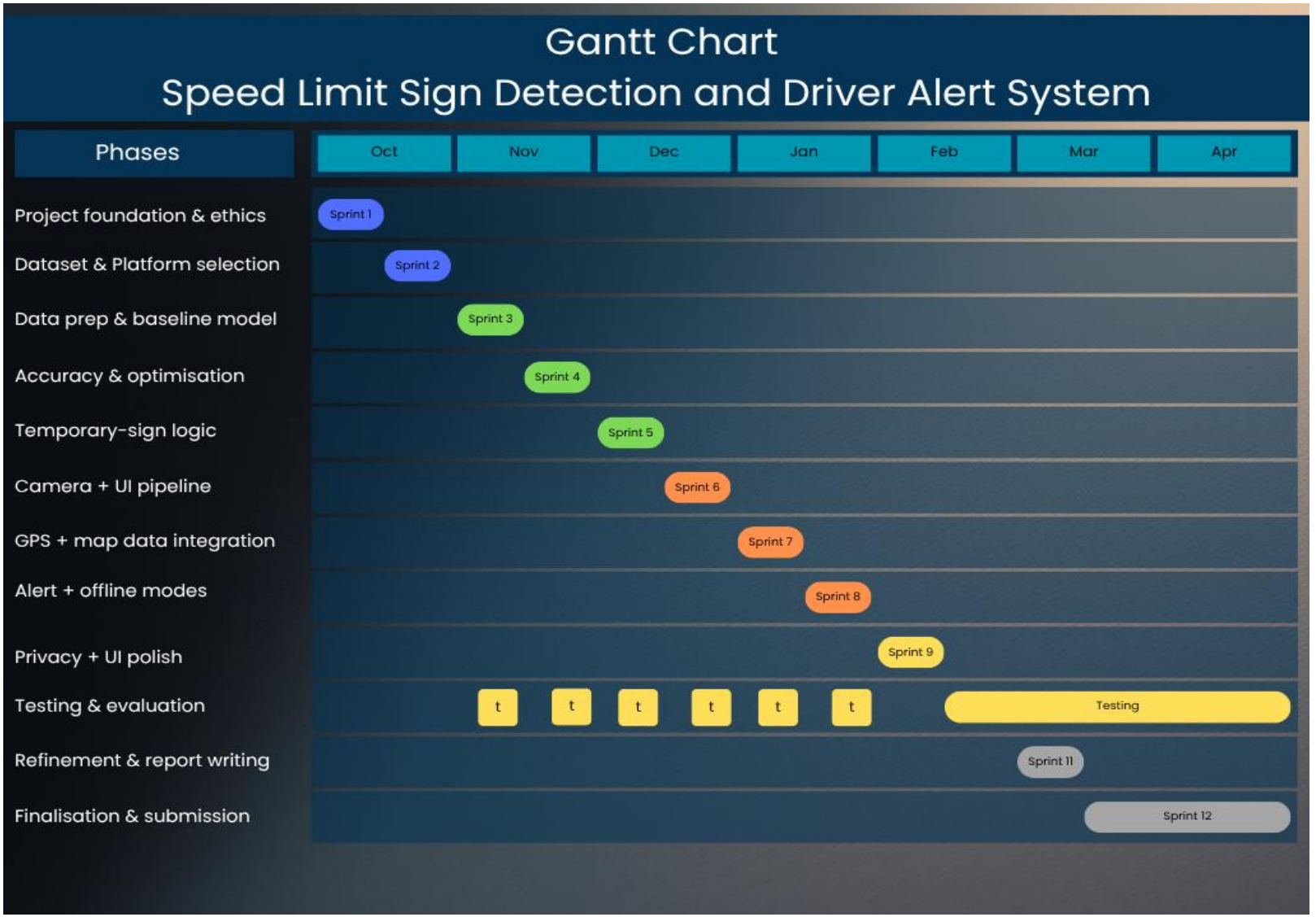
- **Detection accuracy:** Detection accuracy must achieve **≥95% correct value recognition** for standard daylight conditions and **≥90% under night or adverse weather**. False positive rate for non-speed signs must remain **≤1%**.
- **Inference time:** the system must detect and classify a visible speed sign within 1 second of it appearing in view.
- **Alert reliability:** accuracy of mismatch and overspeed alerts.
- **Usability feedback:** clarity of display and alerts in controlled tests.

Testing will use fixed-distance printed signs and varied lighting conditions.

4. References

- GTSRB Dataset – *German Traffic Sign Recognition Benchmark.*
- TSR-UK Dataset (Kaggle).
- Bochkovskiy et al., *YOLO: You Only Look Once*, arXiv.
- Google Maps Platform Documentation.
- University of Hull Ethics and GDPR Framework.

5. Appendix a


Gantt Chart
Speed Limit Sign Detection and Driver Alert System
Phases
Oct
Nov
Dec
Jan
Feb
Mar
Apr
Project foundation & ethics
Sprint 1
Dataset & Platform selection
Sprint 2
Data prep & baseline model
Sprint 3
Accuracy & optimisation
Sprint 4
Temporary-sign logic
Sprint 5
Camera + UI pipeline
Sprint 6
GPS + map data integration
Sprint 7
Alert + offline modes
Sprint 8
Privacy + UI polish
Sprint 9
Testing & evaluation
t
t
t
t
t
t
Testing
Refinement & report writing
Sprint 11
Finalisation & submission
Sprint 12